# V1 to V2 Database Migration Workflow

Dynamic Legacy Freeze and ORM-Based History Backfill

Lex App Migration Notes

February 18, 2026

## 1 Purpose

This document describes the end-to-end migration workflow implemented in `lex/full_migration_workflow.s` and why each step is required. The goal is to migrate a database created by a V1 project into a V2 project while:

- preserving V2 business tables and application behavior,
- freezing V1-only tables as read-only legacy data,
- and backfilling V2 bitemporal history through backend ORM/signal paths.

## 2 Supported Invocation Modes

The workflow now supports two positional-argument modes:

### Mode A: External V1 source provided

```
./lex/full_migration_workflow.sh <V1_MIGRATIONS_SOURCE> <V2_PROJECT_ROOT> <DB_NAME> \
  [--migration-timestamp <ISO8601>] [--chunk-size <INT>] [--dry-run-backfill]
```

Use this when V1 migration files are in a separate V1 repository.

### Mode B: No V1 source provided

```
./lex/full_migration_workflow.sh <V2_PROJECT_ROOT> <DB_NAME> \
  [--migration-timestamp <ISO8601>] [--chunk-size <INT>] [--dry-run-backfill]
```

Use this when V1 migration files are already present in the V2 project's `migrations/` folder. In this mode, the script does not copy migrations; it rewrites the existing files in place and continues normally.

## 3 Workflow Steps and Why They Are Necessary

### 1. Capture pre-migration DB tables

Command: `lex capture_db_tables -output .lex_tables_before.json`

Why: this snapshot is the objective baseline for detecting V1-only tables after schema changes. Without it, freeze decisions are guessed from model code only, which can miss legacy physical tables.

### 2. Prepare migration files

- Mode A: copy V1 migration files into V2 `migrations/`.
- Mode B: use the already-present V2 `migrations/` files directly.

Why: V2 must execute the historical migration chain to preserve schema continuity. If files are already in V2, copying is unnecessary and can overwrite intended state.

### 3. Rewrite V1 migrations for V2 compatibility

Command: `python lex/fix_v1_migration.py <migrations_dir>`

Why: V1 migration imports and model references may target old modules/classes that do not exist in V2. Rewriting makes legacy migrations executable in the V2 runtime.

### 4. Generate pre-migrate freeze manifest

Command: `lex generate_legacy_freeze_manifest ...pre_migrate.json`

Why: this captures the freeze target list before `makemigrations` and migrate alter table operations. It is then used to sanitize newly generated migrations so V1-only tables are not dropped during migration.

### 5. Run `makemigrations`

Command: `lex makemigrations <app>`

Why: V2 model definitions can differ from V1. New migration files encode required V2 schema changes.

### 6. Sanitize generated migrations

Command: `python lex/sanitize_v2_migrations.py ...`

Why: default generated migrations may include destructive operations (e.g., deleting models/tables no longer represented in V2). Sanitization removes operations that would destroy tables intended for legacy freeze.

### 7. Apply migrations

Commands:

```
lex migrate
```

Why: applies all pending migrations across installed apps in dependency order. This avoids missing cross-app dependencies and gives a single deterministic migration entry point.

### 8. Generate final legacy freeze manifest

Command: `lex generate_legacy_freeze_manifest -output .lex_legacy_freeze_manifest.json`

Why: this is the runtime manifest consumed by legacy-data registration. It declares which physical tables should be exposed read-only in admin/API after migration.

### 9. Backfill bitemporal history through ORM

Command:

```
lex backfill_bitemporal_history --chunk-size 500 --reason "V1 migration snapshot" \
  [--timestamp <ISO8601>] [--dry-run]
```

Why ORM/signal path is necessary:

- It reuses tested business logic for history/meta creation and chaining.
- It avoids direct SQL inserts that could bypass invariants and produce invalid state.
- It allows one shared migration timestamp, producing a deterministic historical baseline.
- It is idempotent by skipping models that already have history/meta rows.

## 4 Why Freeze Is Necessary

Some V1 tables are still needed for reporting, audit, or reference, even if V2 has no active model for them. Deleting them loses business context. Keeping them writable is risky because V2 has no full domain behavior around those tables. Read-only dynamic registration is the safe middle ground: preserve access, block mutation.

## 5 Operational Guarantees

- Determinism: one explicit workflow with predictable artifacts and summary output.
- Safety: pre-snapshot + sanitization reduce accidental data loss.
- Compatibility: V1 migration chain can run under V2 after rewrite.
- Auditability: shared timestamp and backfill reason provide a clear migration boundary.
- Re-runnability: backfill skip behavior prevents duplicate history seeding.

## 6 Artifacts Produced

- `.lex_tables_before.json`: physical table snapshot before migration.
- `.lex_legacy_freeze_manifest.pre_migrate.json`: freeze list used for migration sanitization.
- `.lex_legacy_freeze_manifest.json`: final runtime freeze manifest.
- Workflow summary block: machine-readable output for CI/log ingestion.

## 7 Recommended Command Examples

### External V1 source

```
./lex/full_migration_workflow.sh \
  ~/LUND_IT/test/ArmiraCashflowDB \
  ~/LUND_IT/ArmiraCashflowDB \
  db_armiracashflowdb \
  --migration-timestamp "2026-02-18T14:30:00Z" \
  --chunk-size 500
```

### V1 migrations already inside V2

```
./lex/full_migration_workflow.sh \
  ~/LUND_IT/ArmiraCashflowDB \
  db_armiracashflowdb \
  --migration-timestamp "2026-02-18T14:30:00Z" \
  --chunk-size 500
```